When Programs Go Wrong — Errors, and What They Tell You

Class 3 — In-Class Work-Along Sheet (working title · built iteratively)

Name: _____ Section: _____ Date: _____

How to use this sheet: For every program below, **write your prediction FIRST**, *then* run it. Being wrong after thinking hard is exactly the point — a wrong prediction you then correct sticks far better than a right answer you were handed. Keep your **Terminal** open the whole time.

Part 1 — Two Programs, Two Kinds of Crash

Q1. *(Predict, then run.)* Type **Program A** into a file and run it.

```
print("Starting up the program...")
print("Doing some setup...")
print "Hello!"
print("All done.")
```

(a) **Predict first** — what will appear on screen? _____

(b) Now run it. Did **"Starting up the program..."** appear? **Yes / No**

(c) Copy the **last line** of the error you got: _____

(d) What **kind** of error does Python name it (the first word)? _____

Q2. *(Predict, then run.)* Now **Program B**:

```
x = 5
print("x is", x)
y = 7
print("y is", y)
x = "hello"
print("now x is", x)
print(x + 10)
print("This line never runs")
```

(a) **Predict** — what prints, and where (if anywhere) does it break? _____

(b) Run it. **Which lines printed** before it stopped? _____

(c) The error's last line: _____

(d) What **kind** of error is it (first word)? _____

Q3. *(The big one.)* Compare **how the two programs failed**. What is the key difference in *how much of each program ran* before it stopped?

_____

_____

Part 2 — Naming It: *Compile-time* vs *Runtime*

Q4. From Q3: **when** did each program fail? *(Circle one each.)*

- Program A failed: **before any line ran** / **partway through running**
- Program B failed: **before any line ran** / **partway through running**

Q5. *(After we discuss it — books closed.)* In your own words:

- A **compile-time error** is one that _____
- A **runtime error** is one that _____

Q6. Fill the rule:

- In Python, a **compile-time error** is almost always about _____ (bad grammar / spelling-of-a-name / running out of memory).
- **Everything else** that goes wrong only *while the program runs* is a _____ error.

Q7. *(The sneaky one — predict, then run.)* This *looks* like Program A (a `print` typo). Predict: will it behave like A (**nothing runs**) or like B (**some lines run, then crash**)?

```
print("Starting up the program...")
print("Doing some setup...")
Print("Hello!")
print("All done.")
```

(a) **Prediction:** like A / like B — because _____

(b) Run it. Which was it? _____ What error **kind**? _____

(c) In one line: why is `print "Hello!"` (Q1) a *different kind* of error from `Print("Hello!")` here?

_____

Part 3 — Using a Name That Isn't There

Q8. *(Predict, then run.)*

```
print("Starting...")
print(score)
print("This never runs")
```

(a) Error's last line: _____ (b) Kind: _____ (c) Compile-time or runtime? _____

Q9. *(Predict, then run — careful with this one.)* The line `age: int` *looks* like a C++-style declaration ("make me an integer named `age`"). Predict what `print(age)` does:

```
print("Starting...")
age: int
print(age)
```

(a) **Prediction:** _____

(b) Run it. What actually happened? _____

(c) Were you surprised? **Yes / No** — why? _____

Q10. *(The idea — books closed.)* Complete:

> In Python, naming a variable (even with a type, like `age: int`) but **never assigning it a value** is as good as _____, because Python does **not** _____ for a name until you actually _____ it.

Part 4 — Why C++ Catches These *Earlier* (instructor demo — watch & record)

Q11. Your instructor runs the **same two mistakes in C++**: (i) using an undeclared name, and (ii) adding a string and an integer. Watch carefully and fill the table — *especially the "Starting up..." line*:

	PYTHON	C++
Did "Starting up..." print?		
Error caught at... (compile-time or runtime?)	compile / run	compile / run

Q12. *(The why — the heart of today.)* C++ can catch an undefined name (and a type mismatch) at **compile time**, but Python only notices at **runtime**. Why? Write the reason in your own words:

In C++, at compile time the compiler reserves a _____ for each name and ties it to a fixed _____. In Python, a name is just looked up in a _____ **while the program runs**, and a name is **not** tied to any _____.

Part 5 — Dividing by Zero (and a true story)

Q13. *(Predict, then run.)*

```
print("Calculating the class average...")
total = 100
count = 0
print(total / count)
print("Done")
```

(a) **Prediction:** _____

(b) Error's last line: _____

(c) Kind: _____ Compile-time or runtime? _____

(d) After the **USS Yorktown** story: in one line, **why** does Python *refuse* to give an answer here instead of just guessing one?

_____

Part 6 — Go Find More (mini-research · AI allowed)

Q14. Hunt down **two errors we did *not* see today — one compile-time (syntax)** error and **one runtime** error. For each, write: the tiny program, the error's last line, and which **kind** it is.

1) COMPILE-TIME (syntax):
2) RUNTIME:

Q15. *(Recall — books closed, cumulative.)* List **every error type** you met today, and tag each as **compile-time** or **runtime**:

```
+-----------------------------+-----------------------------+
|  ERROR TYPE                 |  COMPILE-TIME or RUNTIME?   |
+-----------------------------+-----------------------------+
|                             |                             |
+-----------------------------+-----------------------------+
|                             |                             |
+-----------------------------+-----------------------------+
|                             |                             |
+-----------------------------+-----------------------------+
|                             |                             |
+-----------------------------+-----------------------------+
```

Part 2 — The Remainder You Can Already Compute

Q16. *(Build it yourself — no new tools.)* I give you two whole numbers, `x = 35` and `y = 19`. Using **only** what you already know (`+ - * /` and `int()`), how would you print the **remainder** when 35 is divided by 19? Write your attempt — being stuck is fine, *try*.

```
+-------------------------------------------------------------------------+
|                                                                         |
|                                                                         |
+-------------------------------------------------------------------------+
```

Q17. *(Trace by hand, then run.)* A student proposed this:

```
x = 35
y = 19
rem = x - int(x/y) * y
print(rem)
```

Fill in each step **by hand** for `x = 35`, `y = 19`:

- `x / y` = _____ → `int(x/y)` = _____ *(how many whole times 19 fits in 35)*
- `int(x/y) * y` = _____ *(the biggest multiple of 19 that fits inside 35)*
- `x - int(x/y) * y` = _____ ← **the leftover = the remainder**

Now run it. Output: _____. In one line, **why** does this give the remainder?

Q18. *(LMS — code it.)* Go to the LMS problem. Initialize two integers, use **this formula** to compute and print the remainder. Then try your **own** two numbers (keep both positive, `x > y`). Does it always match what you'd get by hand?

Q19. *(The shortcut — predict, then run.)* Because programs need remainders constantly, languages give you a one-character tool: the **modulo operator** `%`. Predict each, then run:

```
print(35 % 19)
print(17 % 5)
print(20 % 4)
print(5 % 8)
```

Predictions: _____

Which result surprised you, and why? _____

Q20. *(Why it's everywhere — predict, then run.)* What everyday question does each answer?

```
print(13 % 2)
print(8 % 2)
print(2025 % 10)
print((10 + 5) % 12)
```

- `n % 2` tells you whether a number is _____ or _____.
- `n % 10` gives you the _____ of a number.
- `(10 + 5) % 12` = _____ — modulo **wraps around** (like a clock).

Q21. *(Spot the pattern — predict, then run.)* Predict each, then run:

```
print(12 % 4)
print(12 % 5)
print(12 % 6)
print(12 % 7)
```

(a) Which gave `0`? _____

(b) What is special about the `y` values that gave `0` (versus the others)?

(c) Complete the rule: if `x % y == 0`, then **x is a** _____ **of y** — that is, **y divides x exactly**.

Q22. *(Predict, then run — careful.)* Predict what this does, then run it:

```
print(7 % 0)
```

(a) Prediction: _____

(b) Actual (last line): _____

(c) Which **earlier moment today** does this remind you of, and why? _____

Q23. *(Recall — books closed.)* In your own words, what does `x % y` give you? And complete the formula your hand-written version revealed:

`x % y == x - (` _____ `) * y` *(we'll name that missing piece next part)*

Part 3 — Integer Division `//` (and what negatives do)

Q24. *(Predict the TYPES — then run.)* Predict the **type** each line prints:

```
print(type(5 + 4))
print(type(5 - 4))
print(type(5 * 4))
print(type(5 / 4))
print(type(5 % 4))
```

(a) Which one is **different** from the rest? _____

(b) Fill the rule: `+ - * %` on two ints give an _____; but `/` **always** gives a _____ (even `8 / 2` is `4.0`).

Q25. *(Build it — school division.)* You divide 19 by 7 the way you did as a kid: quotient **2**, remainder **5**. But `19 / 7` gives `2.714…`. Using **only tools you already have**, how would you get **just the quotient** `2`?

Q26. *(The purpose-built tool — predict, then run.)* Python gives you `//` — **floor division** — for exactly this. Predict, then run:

```
print(19 // 7)
print(100 // 7)
print(8 // 0)
```

(a) `19 // 7` = _____ `100 // 7` = _____

(b) `8 // 0` → _____ *(which earlier moment does this echo?)*

Q27. *(Truncate vs floor — predict, then run.)* For **positive** numbers `int(19/7)` and `19 // 7` agree. Now watch a **negative**:

```
print(int(-19 / 7))
print(-19 // 7)
```

(a) `int(-19/7)` = _____ `-19 // 7` = _____ — do they **agree**? _____

(b) `int()` rounds toward _____ (zero / −∞); `//` rounds toward _____.

(c) `-19/7 = -2.714` sits between **−3** and **−2** on the number line. Show where each one lands and why:

```
   ... ---[-4]------[-3]------[ -2.714 ]------[-2]------[-1]------[0] --->
```

Q28. *(The quotient's sign — predict, then run.)*

```
print( 19 //  7)
print(-19 // -7)
print(-19 //  7)
print( 19 // -7)
```

Rule: if the two numbers have the **same** sign, the quotient is _____; if their signs are **different**, the quotient is _____.

Q29. *(The remainder's sign — predict, then run.)*

```
print(-19 % 7)
print(19 % -7)
print(19 / -7, 19 % -7, 19 // -7)
```

(a) `-19 % 7` = _____ `19 % -7` = _____

(b) In `-19 % 7` the quotient is `-3` but the remainder is `+2` — **different** signs! So the remainder does **not** copy the quotient. What **does** it copy? The remainder always takes the sign of the _____ *(numerator / denominator)*.

Q30. *(The conflict — predict both, then run.)* Back in Part 2 you built `rem = x - int(x/y)*y`. Test it against Python's `%` for a **negative**:

```
x, y = -19, 7
print(x - int(x/y) * y)
print(x % y)
```

(a) hand-formula = _____ Python `%` = _____ — do they **agree**? _____

(b) **Why** do they differ? *(Hint: one uses `int()` — truncates; the other uses `//` — floors.)*

(c) Both answers really *are* "a remainder" — under two different world conventions. Python deliberately chose the _____ *(truncate / floor)* convention.

Q31. *(Recall — books closed.)*

- The one identity that is **always** true, for **any** signs: `x == (  _____  ) * y + (  _____  )`.
- So the Part-2 blank — `x % y == x - (____) * y` — the missing piece is _____ *(not `int(x/y)`, which only matches for positive numbers)*.

Part 4 — Division & Modulo on Floats

Q32. *(Predict the TYPES — then run.)*

```
print(type(24 // 6))
print(type(24 // 6.0))
```

(a) `24 // 6` is type _____; `24 // 6.0` is type _____.

(b) Both have **value** 4 — so what decided the **type**? _____

(This is the "widest operand" rule from Part 3, now live.)

Q33. *(Float `/ // %` — predict value AND type, then run.)*

```
print(7.5 / 2)
print(7.5 // 2)
print(7.5 % 2)
```

(a) Values: _____ , _____ , _____ (b) all of type _____

(c) Do the **Part-3 rules** (floor + the leftover) still hold here? **Yes / No** — show why `7.5 // 2` and `7.5 % 2` fit `x == (x//y)*y + (x%y)`:

Q34. *(Why does modulo even work on a decimal? — probe, then run.)* Where would you **ever** want the remainder of a *decimal* division? Think first, then run:

```
print(450.0 % 360.0)
print(26.5 % 24.0)
print(3.75 % 1.0)
```

Match each to what it computes: *wrap an angle into 0–360° · hours on a 24-hour clock · the fractional part.*

- `450.0 % 360.0` = _____ → _____
- `26.5 % 24.0` = _____ → _____
- `3.75 % 1.0` = _____ → _____

Q35. *(Everything at once — predict VALUE and TYPE for each, then run.)*

EXPRESSION	VALUE (guess)	TYPE (guess)
7.5 / 2		
7.5 // 2		
7.5 % 2		
-7.5 // 2		
-7.5 % 2		
7.5 % -2		
24 // 6.0		
-7 // 2		
5.5 % 2.0		

Q36. *(Recall — books closed.)*

- `//` and `%` give an **int** only when _____; if **any** operand is a float, the result is a _____.
- True or false: going from ints to floats changed the **floor** rule and the **sign** rules. _____ *(explain in one line.)*

Part 5 — Who Goes First? Precedence, Power `**`, and Brackets

Q37. *(You already know this — predict, then run.)*

```
print(2 + 3 * 4)
print((2 + 3) * 4)
```

(a) Values: _____ and _____

(b) Why is `2 + 3 * 4` **not** 20? Which school rule decides that `*` happens before `+`?

Q38. *(A new operator: `**` — predict, then run.)* `**` is the **power** operator (`2 ** 3` is "2 to the power 3"). Where does it sit?

```
print(2 ** 3)
print(2 * 3 ** 2)
```

(a) `2 ** 3` = _____ `2 * 3 ** 2` = _____

(b) Is `2 * 3 ** 2` equal to `(2*3)**2 = 36` or `2*(3**2) = 18`? So `**` happens **(before / after)** `*` and `/`.

_____

Q39. *(Why does assignment "just work"? — predict, then run.)*

```
a = 2 + 3 * 4
print(a)
```

(a) `a` = _____

(b) The whole right-hand side is computed **first**, then handed to `a`. That's because `=` has the **lowest** precedence of all — assignment happens _____ *(first / last). (This is why you can put any big expression on the right of `=` and it behaves.)*

Q40. *(Associativity — and the one famous exception. Predict each, then run.)*

```
print(10 - 3 - 2)
print(2 ** 3 ** 2)
print(-2 ** 2)
```

(a) `10 - 3 - 2` = _____ — same-precedence operators run **left to right**, so this is `(10-3)-2`.

(b) `2 ** 3 ** 2` = _____ — surprised? `**` is the **exception**: it runs **right to left**, so it's `2 ** (3 ** 2)` = `2 ** 9`, **not** `(2**3)**2`.

(c) `-2 ** 2` = _____ — another trap: `**` binds **tighter than the minus sign**, so it means `-(2 ** 2)`, not `(-2) ** 2`.

Q41. *(Brackets win — you never have to memorize this.)* Add brackets so each line produces the value shown:

```
# make this equal 20:     2 + 3 * 4
# make this equal 64:     2 ** 3 ** 2
# make this equal 4:      -2 ** 2
```

(a) _____ (b) _____ (c) _____

The golden rule: when in doubt, _____.

Q42. *(Solve by hand first, THEN run to check.)* Let `a = 5` and `b = 2`. Work out each using the ladder; write your by-hand answer, then run it.

```
+----------------------+----------------+----------------+
|  EXPRESSION          |  BY HAND       |  RAN (check)   |
+----------------------+----------------+----------------+
|  2 + 3 * 4 ** 2      |                |                |
+----------------------+----------------+----------------+
|  12 / 2 ** 2 + 1     |                |                |
+----------------------+----------------+----------------+
|  7 // 2 + 7 % 2      |                |                |
+----------------------+----------------+----------------+
|  a + b * a ** b      |                |                |
+----------------------+----------------+----------------+
|  (a + b) * a ** b    |                |                |
+----------------------+----------------+----------------+
|  1.5 * 2 + 1         |                |                |
+----------------------+----------------+----------------+
```

Q43. *(Recall — books closed.)* Fill the precedence ladder from **highest** (goes first) to **lowest**:

```
(  )        _____________________________________
**          power  —  associativity: _________________  (the exception!)
-x          negation
* / // %    multiply / divide  —  associativity: ____________
+ -         add / subtract     —  associativity: ____________
=           assignment  —  precedence is ________ (so the right side runs first)
```

One sentence — when should you reach for brackets? _____

Part 6 — Big Numbers: Why Python Never Overflows

Q44. *(Predict, then run.)* How big a whole number can a computer hold? Predict, then run:

```
print(2 ** 100)
print(2 ** 1000)
```

(a) Did Python complain, round off, or print the **exact** number? _____

(b) `2 ** 100` has _____ digits; `2 ** 1000` has _____ digits *(count them / use a trick)*.

(c) Try an even bigger power. Does Python ever hit a "too big" ceiling? What is the **only** real limit?

_____

Q45. *(Instructor demo — watch C++ and record.)* Your instructor computes factorials (`1! , 2! , … , 21!`) in C++:

```
20! = 2432902008176640000
21! = _____________________________
```

(a) Copy what C++ printed for `21!`: _____

(b) What's **impossible** about it? *(Hint: compare its sign to `20!`.)* _____

(c) Python's `21!` is `51090942171709440000`. The instructor also shows: `largest int + 1` → _____ and `int = 3 billion` → _____

This silent wrap-around to a wrong/negative value is called _____.

Q46. *(Why C++ breaks — fill the idea.)* In C++, when you write `int x`, the computer reserves a **fixed** box of **4 bytes = 32** _____ *(bits / digits)* — **before** it knows your number.

- Even if **every** switch is a 1, that is the _____ number it can hold (~2.1 billion).
- Add one more and it **rolls over** like a car _____ going `999999 → 000000` — except it lands on a large _____ number.

Q47. *(Why Python doesn't — fill the idea.)* Python refuses the fixed box. It stores a big number the way **you** write one on paper — _____ by _____, in a row that **grows** as long as the number needs.

- `2 ** 100` needs 31 slots; `2 ** 1000` needs 302. Need more? Python just _____ more slots.
- The only limit is your computer's _____ *(not a fixed 32 bits).*
- The part of Python that does this growing is the interpreter, _____ *(name it)* — itself a program written in C.

Q48. *(Mini-research — AI allowed.)* Where do **genuinely huge** integers actually matter in the real world? Find **one** concrete example (hint: think about how your messages are kept private online).

```
+-------------------------------------------------------------------------+
|                                                                         |
|                                                                         |
+-------------------------------------------------------------------------+
```

Q49. *(Recall — books closed.)*

- C++ keeps an int in a **fixed-size box**, so it can _____ (wrap to a wrong value).
- Python **grows** an int's storage to fit, so it _____ overflows — the only limit is _____.

Part 7 — Tools Someone Already Built: Built-ins & the `math` Module

Q50. *(Build it, then meet the tool.)* You want **how far `-7` is from zero** — always a positive distance. With only what you know (no `if` yet), one trick is `(-7 * -7) ** 0.5`. Try it:

```
print((-7 * -7) ** 0.5)
print(abs(-7))
print(abs(-3.5))
```

(a) `(-7*-7)**0.5` = _____ (what **type**?) ; `abs(-7)` = _____ (what type?)

(b) In one line, what does `abs()` do? _____

Q51. *(Predict FIRST — you will probably be wrong, that's the point.)*

```
print(round(0.5))
print(round(1.5))
print(round(2.5))
print(round(3.5))
print(round(3.14159, 2))
```

(a) Predict all five: _____

(b) `round(2.5)` is **2**, not 3?! Look at the pattern — `0.5→0, 1.5→2, 2.5→2, 3.5→4`. Python rounds a `.5` to the nearest _____ number. *(This is "banker's rounding" — ask AI why it avoids bias.)*

Q52. *(Two more built-ins — predict, then run.)*

```
print(pow(2, 3))
print(max(3, 7, 2))
print(min(3, 7, 2))
```

(a) `pow(2, 3)` = _____ — which operator from Part 5 does this match? _____

(b) `max(3, 7, 2)` = _____ `min(3, 7, 2)` = _____

Q53. *(Discover something — predict, then run.)* Try to take a square root the obvious way:

```
print(sqrt(16))
```

(a) What happened? _____

(b) Now run this instead:

```
import math
print(math.sqrt(16))
```

(c) `sqrt` isn't a built-in — it lives in a **toolbox** called a **module**. What does `import math` do, and what does the **dot** in `math.sqrt` mean?

Q54. *(Open the `math` drawer — predict, then run.)*

```
import math
print(math.pi)
print(math.floor(2.7), math.ceil(2.1))
print(math.factorial(21))
print(math.gcd(24, 36))
```

(a) `math.floor(2.7)` = _____ — which **operator** from Part 3 does `floor` remind you of? _____

(b) `math.factorial(21)` = _____ — recall Part 6: in C++ this **overflowed to negative**. Here it's _____.

(c) `math.gcd(24, 36)` = _____

Q55. *(Apply — straight-line programs.)* Use the tools to compute:

```
import math
# (a) area of a circle of radius 5:
print(math.pi * 5 ** 2)
# (b) the long side of a right triangle with legs 3 and 4:
print(math.sqrt(3 ** 2 + 4 ** 2))
```

(a) area = _____ (b) hypotenuse = _____

Q56. *(Recall — books closed.)*

- A **built-in** (like `abs`, `round`, `max`) is available _____ *(always / only after import).*
- A **module** (like `math`) must first be _____, then you call its tools with a _____ *(e.g. `math.sqrt`).*
- One reason Python keeps `sqrt` in a module instead of built-in:
  _____

Part 8 — The `random` Module: Can a Computer Surprise You?

Q57. *(Roll the dice — run it several times.)*

```
import random
print(random.randint(1, 6))
```

(a) Run it **5 times**. Write what you got each time: _____ _____ _____ _____ _____

(b) What changes between runs? _____ What is the smallest and largest it ever gives? _____

Q58. *(Two more — run each a few times.)*

```
import random
print(random.random())
print(random.choice(["Aarav", "Diya", "Kabir", "Meera"]))
```

(a) `random.random()` gives a number between _____ and _____ *(a decimal).*

(b) `random.choice(...)` does what? _____ *(the `[...]` is a **list** — a collection in square brackets; you'll learn lists soon.)*

Q59. *(The big one — predict FIRST, then run.)*

```
import random
random.seed(42)
print(random.randint(1, 6), random.randint(1, 6), random.randint(1, 6))
random.seed(42)
print(random.randint(1, 6), random.randint(1, 6), random.randint(1, 6))
```

(a) **Predict:** will the two lines print the **same** three numbers or **different** ones? _____

(b) Run it. What actually happened? _____

(c) So is the computer **truly** random? If `seed(42)` always gives the same sequence, then "random" is really _____ *(a word starting with "pseudo")* — it follows a fixed formula from a starting **seed**.

Q60. *(Why this matters — mini-research, AI allowed.)* Why would an **AI / machine-learning** engineer deliberately set a `random.seed(...)` before training a model? *(Hint: think about a teammate trying to get the **exact same result** as you.)*

```
+-------------------------------------------------------------------------+
|                                                                         |
|                                                                         |
+-------------------------------------------------------------------------+
```

Q61. *(Recall — books closed.)*

- To use `randint`, `random`, `choice`, you first _____ the `random` module.
- A computer's "random" numbers are really _____ — generated by a formula.
- Setting the same _____ before generating makes you get the **exact same** numbers again (vital for **reproducible** experiments).